

P1301R4

[[nodiscard("should have a reason")]]

Published Proposal, 2019-07-16

Authors:

[JeanHeyd Meneide](#)

[Isabella Muerte](#)

Audience:

CWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Latest:

https://theohd.github.io/vendor/future_cxx/papers/d1301.html

Reply To:

[@slurpsmadrips](#)

Abstract

This paper proposes allowing an explanatory string token for use with the `nodiscard` attribute.

Table of Contents

1 Revision History

- 1.1 Revision 4 - July 15th, July 16th, 2019
- 1.2 Revision 3 - June 17th, 2019
- 1.3 Revision 2 - January 21st, 2019
- 1.4 Revision 1 - November 7th, 2018
- 1.5 Revision 0 - October 7th, 2018

2 Feedback

3 Motivation

4 Design Considerations

5 Wording

- 5.1 Intent
- 5.2 Feature Test Macro
- 5.3 Proposed Wording

References

Informative References

§ 1. Revision History

§ 1.1. Revision 4 - July 15th, July 16th, 2019

- Preliminary Core Working Group suggestions from Jens Maurer about basic source character set applicability.
- Rebase on n4820.

— Basically copy-pasta the updated `[[deprecated]]` warning.

§ 1.2. Revision 3 - June 17th, 2019

- Wording is now relative to [\[n4820\]](#), the latest C++ Standard working draft.
- Approved for Core in Kona, going there! 🎉

§ 1.3. Revision 2 - January 21st, 2019

- Wording is now relative to [\[n4778\]](#), the latest C++ Standard working draft.
- Add [§2 Feedback](#) from reflector discussion.

§ 1.4. Revision 1 - November 7th, 2018

- Add Tony Table and fix the example in the Tony Table.

§ 1.5. Revision 0 - October 7th, 2018

- Initial release.

§ 2. Feedback

After a brief discussion with no negative feedback on November in San Diego, the chair forwarded this to EWG with the caveat that this paper should ask both LWG and LEWG representatives to chime in. A message was sent out on the reflector [with 50+ responses indicating strong support](#) (90%+ positive affirmation) for having this feature.

The question is whether or not the standard should mandate the text that goes in a `[[nodiscard]]` or `[[deprecated]]` attribute. There was extremely strong push back to that idea, and this paper does not propose any standardized text on any entity within the C++ Standard. This paper encourages implementers to apply any text they see fit, and if they find it overwhelmingly useful to bring that information in a new and separate paper.

This paper simply proposes making it possible to have the reason present, nothing more.

§ 3. Motivation

Currently

```

struct data_holder {
private:
    std::unique_ptr<char[], data_deleter> ptr;
    std::vector<int> indices;
    // ...

public:
    [[nodiscard]]
    char* release() { indices.clear(); return ptr.release(); }

    void clear() { indices.clear(); ptr.reset(nullptr); }

    [[nodiscard]]
    bool empty() const { return ptr != nullptr && !indices.empty(); }
};

int main () {
    data_holder dh = /* ... */;

    // serious error,
    // do not (void) this
    dh.release();
    // just as noisy as release(),
    // but a little less serious
    dh.empty();

    return 0;
}

```

```

struct data_holder {
private:
    std::unique_ptr<char
    std::vector<int> in
    // ...

public:
    [[nodiscard("Possib
    char* release() { i

    void clear() { indi

    [[nodiscard("Did yc
    bool empty() const

};

int main () {
    data_holder dh = /*

    // serious error, w
    dh.release();
    // less serious, wi
    dh.empty();

    return 0;
}

```

The `[[nodiscard]]` attribute has helped prevent a serious class of software bugs, but sometimes it is hard to communicate exactly why a function is marked as `[[nodiscard]]`.

This paper adds an addendum to allow a person to add a string attribute token to let someone provide a small reasoning or reminder for why a function has been marked `[[nodiscard("Please do not leak this memory!")]]`.

§ 4. Design Considerations

This paper is an enhancement of a preexisting feature to help programmers provide clarity with their code. Anything that makes the implementation warn or error should also provide some reasoning or perhaps point users to a knowledge base or similar to have any questions they have about the reason for the `nodiscard` attribute answered.

The design is very simple and follows the lead of the `deprecated` attribute. We propose allowing a string literal to be passed as an attribute argument clause, allowing for `[[nodiscard("You must use the returned token with api::foobar")]]`. The key here is that there are some `nodiscard` attributes that have different kinds of "severity" versus others. That is, the cost of discarding `vector::empty`'s return value is probably of slightly less concern than `unique_ptr::release`: one might manifest as a possible bug, the other is a downright memory leak.

Adding a reason to `nodiscard` allows implementers of the standard library, library developers, and application writers to benefit from a more clear and concise error beyond "Please do not discard this return value". This makes it easier for developers to understand the intent of the code they are using.

§ 5. Wording

All wording is relative to [\[n4820\]](#).

§ 5.1. Intent

The intent of the wording is to let a programmer specify a string literal that the compiler can use in their warning / error display for, e.g. `[[nodiscard("Please check the status variable")]]`. Multiple redeclarations leave it up to the implementation what happens to the diagnostic message, but does not require them to define and document such behavior.

§ 5.2. Feature Test Macro

`nodiscard` already has an attribute value in Table 15. If this proposal is accepted, then per the standard's reasoning the value of `nodiscard` should be increased to some greater value (such as 201907L).

§ 5.3. Proposed Wording

Modify §15.1 [cpp.cond]'s Table 15:

Attribute	Value
nodiscard	201603L 201907L

Modify §9.11.9 Nodiscard attribute [**dcl.attrnodiscard**]'s clause 1 as follows:

¹ The *attribute-token* `nodiscard` may be applied to the *declarator-id* in a function declaration or to the declaration of a class or enumeration. It shall appear at most once in each *attribute-list* ~~and no attribute-argument clause shall be present.~~
~~. An attribute-argument clause may be present and, if present, it shall have the form:~~

~~(*string-literal*)~~

² A name or entity declared without the `nodiscard` attribute can later be redeclared with the attribute and vice-versa. [Note: Thus, an entity initially declared without the attribute can be marked as `nodiscard` by a subsequent redeclaration. However, after an entity is marked as `nodiscard`, later redeclarations do not remove the `nodiscard` from the entity. — end note] Redclarations using different forms of the attribute (with or without the *attribute-argument clause* or with different *attribute-argument clauses*) are allowed.

³ [Note: A `nodiscard` call is a function call expression that calls a function previously declared `nodiscard`, or whose return type is a possibly cv-qualified class or enumeration type marked `nodiscard`. Appearance of a `nodiscard` call as a potentially-evaluated discarded-value expression is discouraged unless explicitly cast to `void`. Implementations should issue a warning in such cases. This is typically because discarding the return value of a `nodiscard` call has surprising consequences. The *string-literal* from a `nodiscard` attribute-argument clause is intended to be used in the message of the warning as the rationale for why the entity must not be discarded or to suggest alternative entities that may demonstrate the intent or proper usage. — end note]

§ References

§ Informative References

[N4778]

ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. [N4778 - Working Draft, Standard for Programming Language C++](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4778.pdf), November 26th, 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4778.pdf>

[N4820]

ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. [n4820 - Working Draft, Standard for Programming Language C++](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4820.pdf), January 21st, 2019. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4820.pdf>